

Caderno de Arquitetura

Finalidade

Este documento regista a arquitetura proposta para o sistema Wiigu. O objetivo é explicar os elementos principais da aplicação, os seus relacionamentos, as decisões de desenho técnico (*design decisions*), as restrições consideradas, os mecanismos arquiteturais e o impacto das tecnologias escolhidas para o protótipo.

Contexto

O Wiigu é uma aplicação *web* de apoio à gestão visual de projetos utilizando a metodologia Kanban. O sistema permite autenticar utilizadores, organizar projetos, estruturar quadros, criar raias (*swimlanes*), registar cartões, mover cartões entre colunas e raias, definir limites de WIP (*Work in Progress*) e calcular métricas ágeis.

Cada quadro Kanban instanciado no protótipo deve possuir intrinsecamente as colunas obrigatórias: **A FAZER**, **FAZENDO** e **FEITO**.

Objetivos Arquiteturais

- Entregar um protótipo *web* funcional e demonstrável.
- Assegurar a clara separação entre Apresentação, Regras de Negócio e Persistência de Dados.
- Manter uma estrutura enxuta, estritamente adequada ao escopo académico.
- Permitir a rastreabilidade ponta a ponta: das funcionalidades aos requisitos, ecrãs, banco de dados e roteiros de testes.
- Registar persistentemente o histórico de movimentações dos cartões para viabilizar o cálculo rigoroso das métricas Kanban.

Suposições

- O protótipo será executado num ambiente local (ou servidor de desenvolvimento simples) para fins de demonstração académica.
- Os utilizadores acederão ao sistema exclusivamente através de navegadores *web* modernos.
- O volume de dados transacionado será reduzido, visto que o foco é validar a integração dos serviços obrigatórios.

- A segurança aplicar-se-á num nível básico, suficiente para garantir o registo, a autenticação e a integridade da sessão.
- O sistema será autónomo e não dependerá de integrações externas para cumprir os requisitos fundamentais.
- O *login* federado via Google OAuth é tratado como uma integração opcional, habilitada apenas mediante a configuração prévia de credenciais no ambiente.
- A equipa utilizará um sistema de controlo de versões (Git) para gerir os artefactos de documentação e o código-fonte.

Dependências Consideradas

- **Requisitos Funcionais e Histórias de Utilizador:** Definem os serviços de valor que a arquitetura deve necessariamente suportar.
- **Requisitos Não Funcionais:** Impõem as restrições de usabilidade, desempenho, segurança, persistência e as normas documentais.
- **Projeto Físico de Banco de Dados:** Mapeia as entidades, chaves e relacionamentos relacionais consumidos pela camada de persistência.
- **Projeto de Interface (UX):** Define os fluxos de navegação e as componentes visuais que irão interagir com a API de regras de negócio.
- **Roteiro de Testes:** Orienta os cenários de uso que a infraestrutura e a arquitetura precisam sustentar sem falhas durante a avaliação.

Requisitos Arquiteturalmente Significativos

- Autenticação obrigatória de utilizadores para acesso a rotas privadas e serviços do domínio.
- Persistência estruturada e relacional das entidades principais (utilizadores, projetos, quadros, raia, colunas, cartões e movimentações).
- Inicialização automática e sistémica das colunas **A FAZER**, **FAZENDO** e **FEITO** na criação de qualquer quadro.
- Registo imutável de eventos de movimentação para fundamentar o cálculo de *Lead Time*, *Cycle Time*, *Throughput* e estado atual do *WIP*.
- Processamento e validação de bloqueio do limite *WIP* por coluna.
- Interface gráfica baseada em componentes web reativos, com suporte a ações diretas (incluindo *drag-and-drop*).

Decisões Arquiteturais

DA-01: Aplicação baseada em ambiente Web

- **Decisão:** Implementar o Wiigu como uma aplicação *web* (arquitetura cliente-servidor).
- **Justificativa:** A execução via navegador facilita a demonstração, elimina a necessidade de instalação de *software* nas máquinas dos avaliadores e adequa-se perfeitamente aos requisitos do protótipo.

DA-02: Separação em Camadas Lógicas

- **Decisão:** Estruturar o sistema baseando-se nas camadas lógicas de Apresentação (*Frontend*), Regras de Negócio (API) e Persistência (*Database*).
- **Justificativa:** Esta separação diminui o acoplamento sistêmico e demonstra a aplicação prática de conceitos de Engenharia de Software. A apresentação foca-se em UI/UX; a API centraliza as validações (WIP, movimentações, métricas); e a persistência isola o motor de dados.

DA-03: Stack Tecnológica do Protótipo

- **Decisão:** Adotar a *stack* composta por *Frontend* em React, *Backend* (API RESTful) em Node.js com Express, e persistência em banco SQLite.
- **Justificativa:** Esta combinação assegura agilidade na construção do protótipo integrado. O formato de API REST facilita a comunicação assíncrona. O SQLite foi escolhido por fornecer a robustez relacional exigida (comportando o volume de dados do projeto) sem o peso operacional de configurar um SGBD dedicado (como PostgreSQL ou MySQL) em ambiente de demonstração local.

DA-04: Histórico de Movimentações (Rastreabilidade de Estado)

- **Decisão:** Registrar em tabela própria cada mudança relevante de eixo (coluna ou raia) efetuada num cartão.
- **Justificativa:** O motor de métricas Kanban depende intrinsecamente da temporalidade (datas de criação, início, conclusão e transição). Sem um histórico granular de movimentações, seria impossível calcular corretamente as vazões e os tempos de ciclo.

DA-05: Colunas Obrigatórias Inicializadas pelo Sistema

- **Decisão:** A injeção das colunas **A FAZER**, **FAZENDO** e **FEITO** deve ocorrer de forma automática e programática no momento da criação de um novo quadro no banco de dados.
- **Justificativa:** O documento de visão e os requisitos exigem essa configuração estrutural. Automatizar a criação na camada de domínio evita erros operacionais do utilizador e assegura a integridade do requisito.

DA-06: Arquitetura de Login Google (OAuth) Desacoplada

- **Decisão:** Preparar o fluxo de autenticação federada com o Google como um serviço isolado, habilitado exclusivamente através de variáveis de ambiente (`.env`).
- **Justificativa:** O projeto deve permanecer plenamente funcional em modo *offline* ou local. O fluxo base de autenticação própria (e-mail/senha local) é a garantia principal; o OAuth atua apenas como enriquecimento arquitetural se as chaves forem providenciadas no servidor.

DA-07: Validação de Limite WIP na API (Server-Side)

- **Decisão:** A validação que bloqueia o avanço de um cartão para uma coluna com o limite WIP esgotado deve ser feita pela API de *Backend*.
- **Justificativa:** A camada de segurança e regra de negócio nunca deve residir apenas no *Frontend*. Centralizar a validação no *Backend* assegura que a regra se aplique não importando a via de interação (submissão de formulário, *drag-and-drop* ou eventual chamada direta à API).

Restrições do Projeto

- O desenvolvimento do protótipo está severamente delimitado pelo cronograma e prazo da disciplina acadêmica.
- A interface deve primar pela clareza, evitando complexidades visuais desnecessárias.
- A arquitetura adotada deve garantir que o projeto seja descarregado, compilado e executado localmente de modo simples (ex: uso de instâncias locais ou contentores básicos).
- Estão vedadas integrações externas obrigatórias que possam inviabilizar a avaliação por indisponibilidade de rede ou falha de terceiros.

Mecanismos Arquiteturais

- Gestão de identidade (*Autenticação*) e persistência de sessão (JWT ou *cookies*).
- Validação e sanitização de dados submetidos em formulários e requisições HTTP.
- Mecanismo *trigger* ou transacional para injeção automática de colunas na criação de quadros.
- Padrão *Repository* ou *Data Access Object* (DAO) para o acesso e abstração do SQLite.
- Registo sistemático (*log* de dados) para movimentações de estado dos cartões.
- Motor algorítmico interno para extração de métricas.
- Componentes reativos (React) para verificação condicional e renderização do limite WIP visual.
- Bloqueio HTTP (ex: *Status Code* 400 ou 422) pela API quando ocorre tentativa de violação do limite WIP.

Abstrações Principais (Entidades de Domínio)

- `User` : Representa o utilizador detentor de credenciais e capacidade de autenticação.

- **Project** : Representa o grande agrupamento de escopo de trabalho.
- **ProjectMember** : Relacionamento (NxN) que mapeia a vinculação de utilizadores aos projetos.
- **Board** : Representa um quadro visual (Kanban) pertencente a um projeto.
- **BoardColumn** : Representa os estágios (fases do fluxo) pertencentes a um quadro.
- **Swimlane** : Representa a linha horizontal (raia) para separação contextual dentro do quadro.
- **Card** : Representa a unidade fundamental de valor ou atividade (cartão).
- **CardMovement** : Representa uma transação temporal registada (mudança de coluna/raia do cartão).
- **MetricService** : Serviço especializado de domínio para o cômputo e agregação dos dados de fluxo ágil.
- **AuthService** : Módulo responsável pela gestão de acesso, *tokens* locais, validação de *tokens* federados (Google) e fim de sessão.

Perspetiva Lógica

A aplicação estrutura-se ao redor das entidades do domínio orientadas à gestão visual de atividades. A comunicação dá-se numa relação de composição e dependência.

